Mi Backlog – Biblioteca personal de juegos, series y películas

Problema

¿Qué problema resuelve?
Hoy consumimos contenido en muchas plataformas distintas (Steam, Netflix, Disney+, consolas, etc.) y es fácil perder el registro de qué empezamos, qué terminamos, qué nos recomendaron y qué nos gustó. Las listas terminan dispersas en notas del celular, capturas o en la memoria.

¿Quién la utilizaría?
Cualquier persona que juegue videojuegos o mire series y películas con frecuencia y quiera tener todo en un solo lugar: estudiantes, gamers, fans de series, o alguien que quiere organizar "lo que tengo pendiente".

¿Por qué sería útil?

- Centraliza en una sola lista todo lo pendiente, en curso y terminado, sin importar la plataforma.
- Permite recordar qué nos pareció cada cosa (puntaje y comentario personal).
- Ayuda a decidir qué jugar o mirar a continuación, filtrando por tipo, estado o género.
- Evita volver a empezar algo que ya habíamos visto o abandonado.

Funcionalidades

1. **Agregar un título**: cargar un juego, serie o película nuevo con sus datos.
2. **Editar un título**: modificar cualquier dato (por ejemplo, pasar de "Pendiente" a "En curso" o actualizar el puntaje).
3. **Eliminar un título**: borrar un registro, con confirmación previa.
4. **Buscar**: encontrar títulos escribiendo parte del nombre.
5. **Filtrar**: por tipo (juego / serie / película), estado, plataforma, género y favoritos.
6. **Ordenar**: por nombre, puntaje o fecha de agregado.
7. **Marcar como favorito**: destacar los títulos preferidos con un clic.
8. **Estadísticas simples**: ver cuántos títulos hay en cada estado (ej.: 12 pendientes, 3 en curso, 20 terminados).
9. **Exportar / importar**: descargar la biblioteca en un archivo JSON y volver a cargarla (sirve como copia de seguridad).

Datos

Cada registro (título) tiene la siguiente información:

Campo	Tipo	Obligatorio	Descripción / valores posibles
`id`	texto (único)	Sí (automático)	Identificador generado por la app.
`titulo`	texto	Sí	Nombre del juego, serie o película. Máx. 100 caracteres.
`tipo`	lista	Sí	`Juego`, `Serie`, `Película`.

`estado`	lista	Sí	`Pendiente`, `En curso`, `Terminado`, `Abandonado`. Por defecto: `Pendiente`.
`plataforma`	texto	No	Ej.: Steam, PS5, Switch, Netflix, Disney+, Prime Video.
`genero`	lista	No	Acción, Aventura, RPG, Comedia, Drama, Terror, Ciencia ficción, Deportes, Otro.
`puntaje`	número	No	Del 1 al 10. Solo tiene sentido si el estado es `Terminado` o `Abandonado`.
`comentario`	texto largo	No	Opinión o notas personales. Máx. 500 caracteres.
`favorito`	sí/no	No	Por defecto: no.
`fechaCreacion`	fecha	Sí (automático)	Cuándo se agregó a la biblioteca.
`fechaActualizacion`	fecha	Sí (automático)	Última vez que se modificó.

SPEC

Objetivo
Construir una aplicación web simple que permita registrar y organizar juegos, series y películas en una biblioteca personal, indicando su estado, puntaje y opinión.

Usuario
Una sola persona usando la app en su propio navegador (no hay cuentas ni usuarios múltiples).

Funcionalidades (MVP)

- Alta, edición y baja de títulos (CRUD).
- Búsqueda por nombre.
- Filtros por tipo, estado, plataforma, género y favoritos (combinables).
- Orden por nombre, puntaje o fecha.
- Marcar/desmarcar favorito.
- Contador de títulos por estado.
- Exportar e importar la biblioteca en JSON.

Restricciones

- Sin backend ni base de datos externa: los datos se guardan en `localStorage` del navegador.
- Sin frameworks ni librerías externas: solo HTML, CSS y JavaScript puro.
- Debe abrirse haciendo doble clic en `index.html` (sin instalar nada ni levantar un servidor).
- Interfaz en español.
- Diseño responsive: tiene que poder usarse en celular y en PC.
- Validación de datos: no se puede guardar un título sin nombre, tipo ni estado; el puntaje solo acepta valores de 1 a 10.
- Fuera de alcance (por ahora): login, sincronización en la nube, buscar portadas o datos en APIs externas.

Tecnología elegida

- HTML5 para la estructura.
- CSS3 (Flexbox / Grid) para el diseño.

- JavaScript (ES6+, módulos) para la lógica.
- `localStorage` para la persistencia.

Prompt profesional

```
Actuá como un desarrollador frontend senior. Quiero construir "Mi Backlog", una
aplicación web para llevar una biblioteca personal de juegos, series y películas.

## Tecnología
- HTML5, CSS3 y JavaScript puro (ES6+). No uses frameworks ni librerías externas
  (nada de React, Bootstrap, jQuery, etc.).
- Persistencia con localStorage.
- La app tiene que funcionar abriendo index.html directamente en el navegador,
  sin servidor ni instalación.

## Modelo de datos
Cada título es un objeto con estos campos:
- id (string, generado automáticamente)
- titulo (string, obligatorio, máx. 100 caracteres)
- tipo ("Juego" | "Serie" | "Película", obligatorio)
- estado ("Pendiente" | "En curso" | "Terminado" | "Abandonado", obligatorio,
  por defecto "Pendiente")
- plataforma (string, opcional)
- genero (uno de: Acción, Aventura, RPG, Comedia, Drama, Terror,
  Ciencia ficción, Deportes, Otro; opcional)
- puntaje (entero de 1 a 10, opcional)
- comentario (string, opcional, máx. 500 caracteres)
- favorito (boolean, por defecto false)
- fechaCreacion y fechaActualizacion (ISO string, automáticas)

## Funcionalidades
1. Formulario para agregar un título nuevo.
2. Editar un título existente (reutilizando el mismo formulario).
3. Eliminar un título, pidiendo confirmación antes.
4. Buscador por nombre que filtre mientras se escribe (sin distinguir
   mayúsculas ni tildes).
5. Filtros combinables por tipo, estado, plataforma, género y "solo favoritos".
6. Ordenar por nombre (A-Z), puntaje (mayor a menor) o fecha de agregado.
7. Botón para marcar/desmarcar favorito desde la lista.
8. Panel con la cantidad de títulos en cada estado.
9. Exportar la biblioteca a un archivo .json e importar desde un .json
   (validando que el archivo tenga el formato correcto).
10. Mensaje amigable cuando la lista está vacía o ningún título coincide
    con los filtros.

## Cómo espero el código
- Estructura de archivos:
    index.html
    css/styles.css
    js/app.js        -> inicialización y eventos
    js/storage.js    -> lectura/escritura en localStorage
    js/ui.js         -> renderizado de la lista, formulario y estadísticas
    js/validation.js -> validación de datos
- Funciones chicas, con una sola responsabilidad y nombres descriptivos.
- Nombres de variables y funciones en español o inglés, pero consistentes.
- Comentarios breves solo donde la lógica no sea obvia.
- Nada de código duplicado ni variables globales sueltas.
- Usar addEventListener (no onclick en el HTML).

## Restricciones importantes
- Validar el formulario antes de guardar y mostrar los errores al lado del
  campo correspondiente (no usar alert para esto).
- Si localStorage está vacío o tiene datos corruptos, la app no debe romperse:
```

```
  debe arrancar con la lista vacía.
- Escapar el texto que ingresa el usuario al mostrarlo (no usar innerHTML con
  datos sin sanitizar).
- Diseño responsive, limpio y legible en celular y PC. Interfaz en español.
- Accesibilidad básica: labels en todos los inputs, botones con texto o
  aria-label, buen contraste.

Antes de escribir código, mostrame un plan breve con la estructura y los pasos
que vas a seguir. Después implementalo paso a paso y, al terminar, explicame
cómo probar cada funcionalidad.
```

Challenge de validación

Antes de aceptar el código generado por la IA, voy a revisar lo siguiente:

1. ¿Funciona correctamente?

☐ Puedo agregar un título de cada tipo (juego, serie, película).
☐ Puedo editar un título y los cambios se ven en la lista.
☐ Al eliminar, pide confirmación y el título desaparece.
☐ La búsqueda encuentra "zelda" aunque esté guardado como "The Legend of Zelda" y también "pelicula" / "película".
☐ Los filtros se pueden combinar (ej.: Juegos + Pendiente + Favoritos).
☐ El orden funciona en los tres criterios.
☐ Las estadísticas por estado coinciden con la cantidad real de títulos.

2. ¿Los datos se guardan correctamente?

☐ Al recargar la página (F5) los títulos siguen estando.
☐ Al cerrar y volver a abrir el navegador los datos persisten.
☐ Exporto a JSON, borro todo, importo el archivo y la biblioteca queda igual.
☐ `fechaCreacion` no cambia al editar; `fechaActualizacion` sí.

3. ¿Hay manejo de errores?

☐ No me deja guardar sin nombre, tipo o estado, y muestra el error en el campo.
☐ No acepta puntaje 0, 11, negativos ni decimales.
☐ Si importo un archivo que no es JSON válido, muestra un mensaje y no borra lo que ya tenía.
☐ Si borro o corrompo manualmente el `localStorage`, la app abre igual sin romperse.
☐ Si escribo `<script>alert(1)</script>` como título, se muestra como texto y no se ejecuta.

4. ¿Respeta la estructura esperada?

☐ Los archivos están separados como pedí (`storage.js`, `ui.js`, `validation.js`, `app.js`).
☐ No usa frameworks ni librerías externas.
☐ Abre con doble clic en `index.html`, sin servidor.

5. ¿Es fácil de entender?

☐ Puedo explicar con mis palabras qué hace cada archivo y cada función principal.
☐ Los nombres son claros y no hay código repetido.
☐ No hay errores ni advertencias en la consola del navegador (F12).

6. ¿La interfaz es clara?

- ☐ Se entiende cómo agregar un título sin necesidad de instrucciones.
- ☐ Se ve bien en celular (probado con el modo responsive de las DevTools).
- ☐ Se distingue a simple vista el estado de cada título (colores o etiquetas).
- ☐ Hay un mensaje cuando la lista está vacía o no hay resultados.

Si algún punto falla, no acepto el código: le explico al agente qué falló, con el paso a paso para reproducirlo, y le pido que lo corrija.